

ProctoEase AI-Powered Online Examination (Technology Day)

Slide 1: Introducing ProctoEase

What?

Technology showcase introduction for the ProctoEase platform.

Why?

To grab audience attention and establish the product identity.

How (Implementation & Thinking)?

Pitched as a multi-tenant, AI-assisted online examination SaaS product targeting technical recruitment and university exams.

Slide 2: The Problem Statement

What?

Identification of academic dishonesty and operational bottlenecks in remote examinations.

Why?

Justifies the market need for the product by highlighting gaps in current manual proctoring and traditional exam platforms.

How (Implementation & Thinking)?

Addresses specific issues like impersonation, secondary device usage, and lack of scalable monitoring.

Slide 3: The ProctoEase Solution

What?

High-level overview of the platform's multi-tenant AI capabilities and core offerings.

Why?

Shows the comprehensive nature of the automated solution provided to address the problem statement.

How (Implementation & Thinking)?

Delivers end-to-end automated monitoring seamlessly integrated with an intuitive exam delivery environment.

Slide 4: Core Architecture Stack

What?

A breakdown of the backend and database technologies forming the application foundation.

Why?

To prove the system's reliability, modern design, and capability to handle thousands of concurrent interactions.

How (Implementation & Thinking)?

Built using FastAPI for async I/O high performance, PostgreSQL 16 for relational integrity, Redis 7 for pub/sub and caching, and deep Docker containerization.

Slide 5: Multi-Tenant B2B Isolation

What?

Explanation of how different organizational clients are separated within the same database.

Why?

Data privacy is a core SaaS requirement; clients must trust that their candidate data cannot bleed into other accounts.

How (Implementation & Thinking)?

Engineered purely at the database level using PostgreSQL Row-Level Security (RLS) linked programmatically to the `tenant\_id` present in every incoming JWT.

Slide 6: Security, Auth & Role-Based Access

What?

Walkthrough of authentication flows and endpoint security.

Why?

To assure stakeholders of data protection and strict authorized access.

How (Implementation & Thinking)?

Utilized securely hashed passwords with bcrypt, generated dual JWT tokens (Access/Refresh), and enforced Role-Based Access Control (Admin vs Recruiter vs Candidate).

Slide 7: Real-Time AI Proctoring Engine

What?

Deep dive into the live monitoring architecture.

Why?

To showcase the most technically complex aspect of the platform that actively prevents violations.

How (Implementation & Thinking)?

Persistent WebSocket connections stream events from the client browser. Rule-based backends catalog frame events such as `tab\_switch`, `fullscreen\_exit`, and `no\_face`.

Slide 8: Risk Scoring Algorithm

What?

Overview of the composite risk assessment logic.

Why?

To explain how raw technical data is translated into a usable, human-readable suspicion metric for recruiters.

How (Implementation & Thinking)?

Implements weighted scores based on event severity (e.g., looking away might have lower weight than multiple faces detected) to flag highly suspicious attempts automatically.

Slide 9: System Observability & Hardening

What?

Highlighting DevOps best practices making the system production-ready.

Why?

Ensures the system does not silently crash or fail during massive concurrent examination workloads.

How (Implementation & Thinking)?

Implemented structured JSON logging with correlation IDs, comprehensive custom error hierarchies, and defensive error handlers to obscure stack traces from end users.

Slide 10: Future Integrations: Code Execution & Plagiarism

What?

A preview of roadmap core features geared toward technical assessments.

Why?

Highlights advanced capabilities that make the platform highly competitive against existing specialized coding platforms.

How (Implementation & Thinking)?

Planned integration with Judge0 for sandboxed code compilation, alongside Abstract Syntax Tree (AST)-based similarity checks to detect code plagiarism.

Slide 11: DevOps, CI/CD & Deployments

What?

An overview of the software delivery pipeline.

Why?

Demonstrates rapid iteration capacity, zero-downtime potential, and professional software engineering lifecycle management.

How (Implementation & Thinking)?

Utilizes multi-stage Docker builds, GitHub Actions pipelines for linting/testing, and strict container health checks before traffic routing.

Slide 12: Summary & Roadmap

What?

Wrapping up the main presentation pillars and opening the floor for discussion.

Why?

Solidifies the core message and transitions to interactive Q&A.

How (Implementation & Thinking)?

Recaps the pillars: Scalable Architecture, Uncompromised Security, and Intelligent Proctoring.